Automatic Attack Scenario Reconstruction with Large Language Models: Threat Intelligence Extraction Strategies and Techniques*

Cheng-Han Shie[1], Che-Wei Lin[1], Yi-En Chang[1] and Chun-I Fan[123]
[1] Department of Computer Science and Engineering, National Sun Yat-sen University, Taiwan
hanhan3927@g-mail.nsysu.edu.tw, yienchang0424@gmail.com, linwebs@linwebs.tw
[2] Information Security Research Center, National Sun Yat-sen University, Taiwan
[3] Intelligent Electronic Commerce Research Center, National Sun Yat-sen University, Taiwan
cifan@mail.cse.nsysu.edu.tw

Abstract

Recently, the application of large language models (LLMs) has been gradually extended to the information security domain to assist security professionals in vulnerability identification, classification, and penetration testing. However, research on reproducing the entire cyber kill chain remains relatively scarce, with few public studies available due to constraints related to commercial interests and technical thresholds. This paper proposes a system architecture that systematically divides the process of attack reproduction into three stages: threat intelligence organization, scenario construction, and attack script generation. The system is built on open-source software, integrating the functions of LLMs and virtual machine construction, allowing users to reproduce attack processes recorded in threat intelligence through a visualized web interface in a unified platform, gaining a deeper understanding of the sequence of attack events. With the aid of an LLM, this paper successfully applies the system to reproduce common vulnerabilities and exposures (CVE) attack scenarios and the attack techniques of advanced persistent threat (APT) organizations. While user intervention remains necessary in certain stages, including supplementing publicly available exploit references and correcting script errors through iterative feedback, the system significantly reduces the time and expertise required compared to traditional manual reproduction workflows.

* Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 21, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

1 Introduction

With the rapid evolution of information technology, cyberattack methods have become increasingly sophisticated. Attackers frequently target critical infrastructure of government agencies and enterprises [1], and their operations (driven by hacker organizations, transnational criminal groups, or even state-level actors) can cause severe disruptions to essential services. According to IBM's 2025 Cost of a Data Breach Report [2], the mean time to identify and contain a breach has decreased to 241 days, a nine-year low and a 17-day reduction from the previous year, largely driven by the extensive adoption of AI and automation in security detection and containment. These evolving attack methods can bypass traditional firewalls and anti-virus software, enabling attackers to gain system control and steal sensitive data, while victimized organizations bear massive incident-handling costs.

To counter these threats, organizations must establish robust cybersecurity defense mechanisms, including regular training to enhance personnel awareness and cultivating dedicated cybersecurity professionals. However, developing such expertise typically requires years of learning and practical experience. Furthermore, cybersecurity researchers must invest significant time interpreting threat intelligence from diverse sources to reconstruct the full picture of an attack. With the development of large language model (LLM) technology, its powerful knowledge integration and natural language processing capabilities can assist this process, potentially reducing analysis work from days or weeks to hours.

This paper proposes a system that utilizes LLMs to systematically organize and analyze threat intelligence. By designing standardized interpretation guidelines, the system enables LLMs to accurately interpret intelligence content and transform the results into automated environment deployment scripts and attack scripts, significantly reducing the technical threshold and complexity of attack reproduction.

1.1 Contributions

This paper proposes a framework that utilizes LLMs to achieve automated processing and analysis of threat intelligence. This framework can effectively extract, organize, and transform threat intelligence into usable information, thereby automatically generating the scripts required for field construction and deployment. The contributions of this paper are as follows:

- **Using LLM to Construct an LLM-Assisted Indicator of Compromise (IoC) Extraction Mechanism**. Threat intelligence obtained through various channels, such as advanced persistent threat (APT) reports from security vendors, Threat Hunting results, and attack methods shared on cybersecurity forums, is imported into the LLM. Through prompts designed by this paper, the LLM can transform unstructured threat intelligence into standardized analysis results. This not only enhances the usability of threat intelligence but also helps security analysts more effectively master the attackers' tactics, techniques, and procedures (TTPs), thereby constructing complete attack chain analysis reports.
- **LLM-Assisted Attack Scenario Construction Mechanism Based on Threat Intelligence**. Using the extracted IoCs as input, combined with prompts designed for virtualization platforms, the LLM generates corresponding environment setup scripts with minimal user guidance. This mechanism allows the system to interface with virtualization platforms and streamline the deployment of the required experimental environment, significantly reducing the time and labor costs for researchers to reproduce attack scenarios.
- **Scenario-Oriented LLM-Assisted Attack Script Generation Mechanism**. Through designed prompts, the LLM is guided to interpret key information in threat intelligence, including attack methods and victim targets. Simultaneously, by providing environment

information such as machine configurations and network architecture within the scenario, the LLM can leverage knowledge learned during its training phase, combined with scenario characteristics and attack requirements, to design automated attack scripts applicable to specific targets.

1.2 Structure

The remainder of this paper is organized as follows. Section 2 reviews related work, followed by an overview of the preliminary technologies in Section 3. Section 4 details the proposed system architecture and the attack reproduction process. In Section 5, we validate the feasibility of our methodology and provide a comparative analysis. Finally, Section 6 concludes the work and discusses future directions.

2 Related Works

2.1 Using Threat Intelligence as a Basis for Attack Identification

Cyber threat intelligence (CTI) plays a pivotal role in modern attack analysis, enabling both the prediction of threat patterns and the reconstruction of adversarial behavioral trajectories. Parmar and Domingo [3] leveraged CTI within the Coalition Warrior Interoperability eXercise (CWIX), utilizing the MITRE ATT&CK framework [4] to decode adversary behaviors and identify TTPs with higher fidelity. Mundt and Baier [5] further advanced this direction by extracting granular elements, such as malware signatures and mitigation strategies, to characterize the technical methodologies of sophisticated groups like APT29 and APT32, demonstrating the value of CTI in automated threat detection and intelligence-led defense.

2.2 Attack Simulation and Reproduction

By simulating adversarial methodologies, researchers can gain comprehensive visibility into attack lifecycles. Karagiannis et al. [6] developed the A-DEMO framework, integrating the MITRE ATT&CK matrix to document malware techniques and analyze execution patterns of Rootkit malware in a simulated medical environment, focusing on command and control (C&C), persistence, and exfiltration behaviors. Iturbe et al. [7] introduced a hybrid methodology combining threat modeling with attack simulation, constructing an Industrial IoT (IIoT) testbed to reproduce CVE-related vulnerability exploitation [8] stemming from misconfigurations. These approaches provide a critical foundation for understanding attacker TTPs and formulating proactive defense strategies.

2.3 Using LLMs to Assist Penetration Testing

Traditional penetration testing is labor-intensive and heavily reliant on human expertise. Recent advancements have demonstrated that LLMs can significantly augment this process. For instance, Happe et al. [9] introduced HackingBuddyGPT, a system designed to assist researchers in uncovering novel attack vectors via LLMs. Similarly, Deng et al. [10] developed PentestGPT, an interactive framework that employs a tripartite architecture: parsing, reasoning, and generation, to conduct autonomous testing. Their evaluation on platforms such as HackTheBox [11] and VulnHub [12] underscores the potential of LLMs in automating complex security assessments.

2.4 Potential of LLMs in Vulnerability Detection and Identification

LLMs have shown significant potential in vulnerability identification. Lin and Mohaisen [13] conducted a comprehensive evaluation of multiple LLMs, including LLaMA-2, Gemma, and GPT-4, for vulnerability detection, finding that while certain models demonstrate strong identification capabilities, performance varies considerably across configurations and programming languages. Lekssays et al. [14] proposed LLMxCPG, integrating Code Property Graphs with LLMs to achieve 15–40% improvements in F1-score over state-of-the-art baselines by reducing code to vulnerability-relevant slices. Sheng et al. [15] further surveyed the field, noting that LLM effectiveness in vulnerability analysis depends heavily on integration with program analysis techniques, structured prompt engineering, and domain-specific fine-tuning—rather than relying solely on the models' inherent capabilities.

3 Background

3.1 Integration of LLMs

3.1.1. Evaluation and Selection of LLMs

Work by Happe et al. [16] demonstrated that small language models cannot handle complex penetration tasks. We therefore evaluated four LLMs, GPT-4o, Claude 3 Opus, Llama3-70B, and Gemini 1.5 Pro, across three core security tasks, as summarized in Table 1. Among these, Llama3-70B failed to meet our requirements for vulnerability identification and attack script drafting. While the remaining three models performed adequately, GPT-4o was selected as the primary engine due to its superior multi-format input support, extended API capabilities including file search and code interpretation, and its Assistant mode that preserves dialogue context across sessions.

Table 1: Evaluation of Large Language Model Capabilities

Language Model	Vulnerability Identification	Virtual Scenario Configuration	Attack Script Drafting
GPT-4o	✓	✓	✓
Claude 3 Opus	✓	✓	✓
Llama3-70B	-	✓	-
Gemini 1.5 Pro	✓	✓	✓

3.1.2. Interfacing with OpenAI API

To ensure system stability and maintainability, our system interfaces with the OpenAI Assistants API through direct RESTful calls using PHP's cURL library, which supports multi-format threat intelligence input and preserves dialogue threads across sessions. Dedicated Pre-processing and Post-processing Modules (detailed in Section 4.2) are incorporated to reduce token usage and improve response times.

3.2 Construction of Virtualization Experimental Scenario

3.2.1. Virtualization Platforms

Since our objective requires a multi-user virtualization environment with customization flexibility, we evaluated four open-source platforms, as summarized in Table 2. VMware vSphere was excluded due to its closed-source nature. OpenNebula was selected as the primary solution based on its user-friendly interface, flexible architecture facilitating system integration, and robust API support. Although OpenStack offers higher functional completeness, its architecture proved too rigid for our customization requirements. CloudStack and Proxmox VE were too simplified for our advanced needs.

Table 2: Comparison of Open-Source Virtualization Platforms

Platform	Key Characteristics	Limitations
OpenNebula	Intuitive management interface; template-based component presentation; contextualization feature for guest VM communication	Smaller community than OpenStack
OpenStack	Modular architecture (Nova, Glance, Cinder, Neutron, Keystone); extensive API support and active community	Highly complex architecture
CloudStack	Simplified management; lightweight architecture suitable for SMEs	Overly simplified functionality
Proxmox VE	Combines KVM and LXC technologies; user-friendly web interface	Limited extensibility for advanced needs

3.2.2. Virtualization Network Architecture

To simulate realistic attack scenarios across multiple network segments, the system utilizes Open vSwitch (OVS) as the core virtual network switch for Layer 2 networking with VLAN isolation. OPNsense serves as the routing and firewall platform between virtual subnets, providing RESTful API support for automated rule management. User access to internal VMs is secured through OPNsense's built-in WireGuard VPN service, establishing encrypted tunnels for RDP and SSH connectivity.

4 System Design

This section presents the system architecture, the methodology for LLM-assisted threat intelligence analysis, and the mechanisms for LLM-assisted attack script generation.

The operational workflow is divided into three primary stages: threat intelligence organization, scenario construction, and attack script generation. As illustrated in Figure 1, the system follows a two-lane architecture. The left lane depicts the three-stage pipeline through which threat intelligence is progressively transformed into actionable attack simulations. Users begin by importing raw reports from diverse sources (e.g., security vendor publications, technical forums, or dark-web intelligence)

into Stage 1, where the IoC Extraction Module identifies structured indicators such as CVE/CWE identifiers, IP addresses, domain names, and APT lifecycle information. These structured outputs then flow into Stage 2, where the Machine Deployment Module provisions virtual machines and networks based on the extracted indicators, and the Vulnerability Environment Installation Module deploys specific vulnerable software versions via auto-generated scripts. Both modules interact with the OpenNebula and OPNsense virtual lab infrastructure for VM creation and script execution. If an installation script fails, the system supports an error feedback loop that resubmits the error message to the LLM for script regeneration. Once the environment is fully provisioned, Stage 3 invokes the Attack Module to synthesize scenario-aware exploit scripts targeting the deployed victims.

The right lane of Fig. 1 shows the shared LLM Engine that serves all three stages. When any module initiates a query (indicated by the dashed arrows labeled "prompt → JSON"), the Question Recommendation Module first suggests the appropriate query type based on the user's current progress. The Pre-processing Module then resolves internal function calls and attaches relevant domain knowledge (e.g., CWE documentation) to enrich the prompt. The assembled prompt is forwarded by the LLM Query Module to the OpenAI GPT-4o Assistants API via cURL HTTP requests. Upon receiving the response, the Post-processing Module validates and repairs the JSON output before rendering it through the analyst's web interface. All scenario data, VM states, and conversation threads are persisted in a MariaDB data store for continuity across sessions.

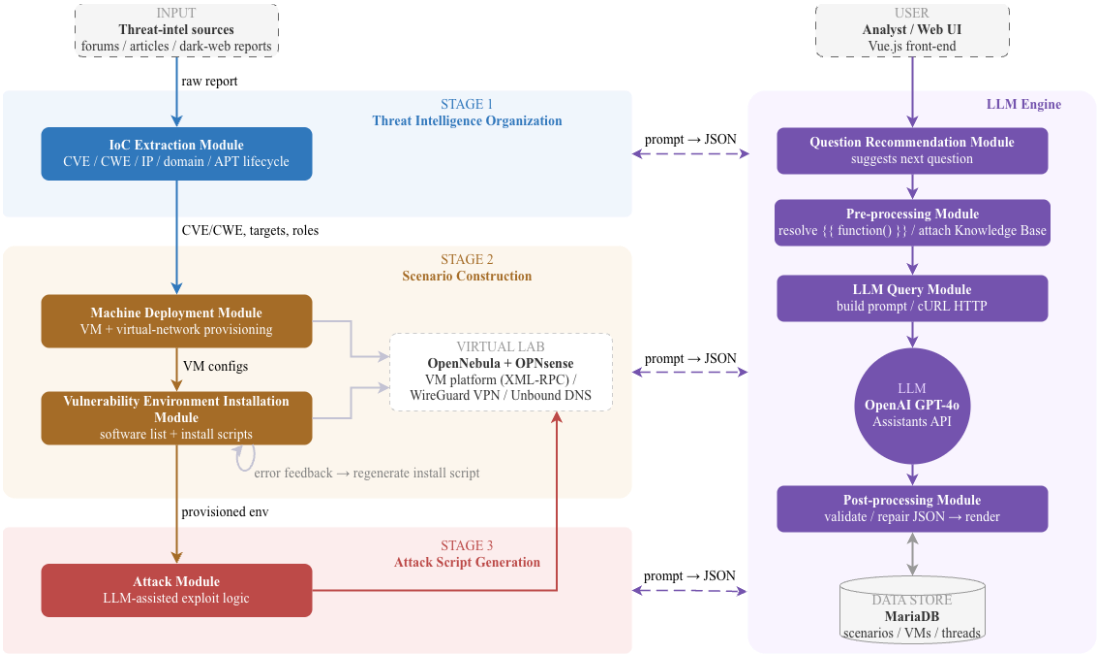


Figure 1: System data flow diagram

4.1 System Architecture

The proposed system is implemented as a web-based application, accessible through any internet-enabled device via a standard web browser. The system architecture leverages Docker containerization for efficient maintenance and updates. The backend utilizes Nginx as the web server, with core application logic developed in PHP using the Laravel framework. MariaDB serves as the primary database, while the frontend is built with Vue.js. For authentication, the system integrates with LDAP to synchronize credentials with the virtualization platform. OpenNebula was selected as the integrated

virtualization engine. The LLM Query Module communicates with the OpenAI GPT-4o API. The entire stack is constructed using freely available open-source software, eliminating licensing costs. The entire environment is composed of VM services and several microservices, utilizing OpenNebula v6.8.0, Laravel v11, PHP v8.3, Nginx v1.26, and MariaDB v11.3.

4.2 Integration with LLMs

The system adopts a modular design, with the LLM Query Module handling the interface to external AI models. By default, the system utilizes GPT-4o, interfacing with the OpenAI API via cURL-based HTTP requests. When a query is initiated, the module transmits structured prompts to the API server and retrieves generated responses for further parsing.

4.2.1. Prompt Engineering Framework

To ensure the LLM accurately understands research objectives and generates high-fidelity responses, we developed a structured prompt engineering framework. As shown in Figure 2, the framework comprises five stages:

- **Context Description.** Establishes the operational background, e.g., "The user needs to construct an environment to simulate an attacker and a victim; please assist in reproducing the vulnerability."
- **Task Specification.** Defines specific requirements, e.g., "Please install the identified vulnerability onto the victim machine within the scenario."
- **Response Format and Exemplar Data.** Provides few-shot exemplars to guide the LLM's response structure and style.
- **Field Descriptions for Response Format.** Defines data types, filling rules, and constraints for each JSON response field to ensure data integrity and consistency.
- **Standard Response Template (Empty).** Provides an unpopulated schema to help the LLM grasp structural relationships.

The ordering of stages three through five can be dynamically adjusted to achieve optimal performance.

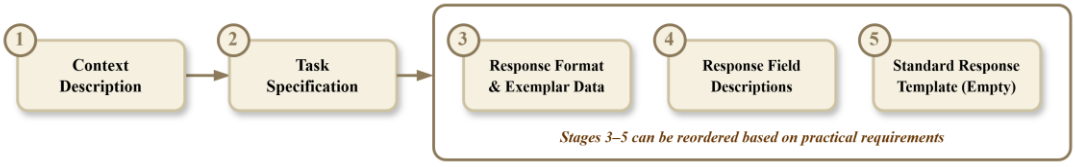


Figure 2: Prompt Engineering Framework

In addition to the core prompting logic, the system incorporates Pre-processing and Post-processing Modules. The LLM Query Module transmits content to these modules before initiating a query and after retrieving a response. Specifically, the Pre-processing Module performs prompt refinement to generate contextually appropriate queries, while the Post-processing Module conducts an initial analysis and validation of the returned results before presenting them to the user.

- **LLM Query Module**

To enable a seamless "one-click" query experience, the module incorporates a comprehensive internal question bank. Users can directly select specific query items, such as identifying CVE numbers from threat intelligence reports, without manually constructing complex prompts. Queries are categorized into four types: vulnerability identification, environment provisioning, vulnerability installation, and attack script synthesis.

The system also features a question recommendation module that suggests contextually relevant follow-up queries based on the user's current progress. For instance, after completing environment provisioning queries, the system automatically guides users toward vulnerability installation options, ensuring even unfamiliar users can successfully deploy environments and achieve LLM-guided attack simulations.

- **Pre-processing Module**

  The system does not transmit raw queries directly to the LLM. As illustrated in Figure 3, the Pre-processing Module retrieves the unprocessed query and systematically parses it to identify function call segments, formatted as `{{function_name}}`. Upon detection, the specified function is executed and its output is merged back into the query. This iterative process continues until all function calls are resolved, ensuring the final prompt is contextually rich and technically precise.

- **Post-processing Module**

  Upon receiving an LLM response, the Post-processing Module verifies content correctness and format integrity before presentation, as illustrated in Figure 4. This module is divided into backend parsing and frontend rendering components.

  Since the system mandates JSON-format responses, but LLM stochasticity means valid JSON is not always guaranteed, the module employs a robust decoding strategy. If a response is not initially valid JSON, the module searches for JSON-like segments delimited by `{and}` and attempts parsing. If parsing still fails, the system applies character transformations to correct erroneous escape characters that LLMs occasionally introduce (e.g., prefixing `$`, `[`, `]` with backslashes). The replacement sequence is defined in Table 3. Once validated, the JSON object is parsed into constituent fields and rendered on the frontend interface, with critical information highlighted for user comprehension.

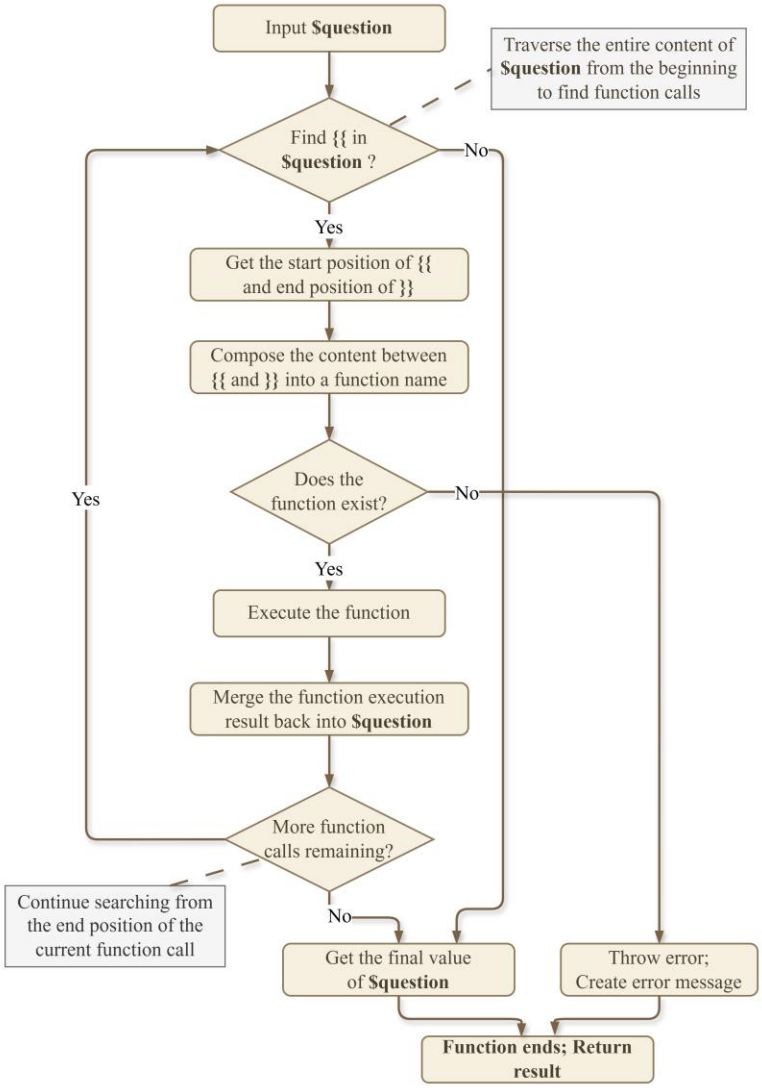


Figure 3: Pre-processing Module

Table 3: LLM Response Escape Character Replacement Table

Replacement Order	Original Character (Pre-replacement)	Normalized Character (Post-replacement)
1	\$	$
2	\\[	[
3	\\]	]
4	\[	[
5	\]	]

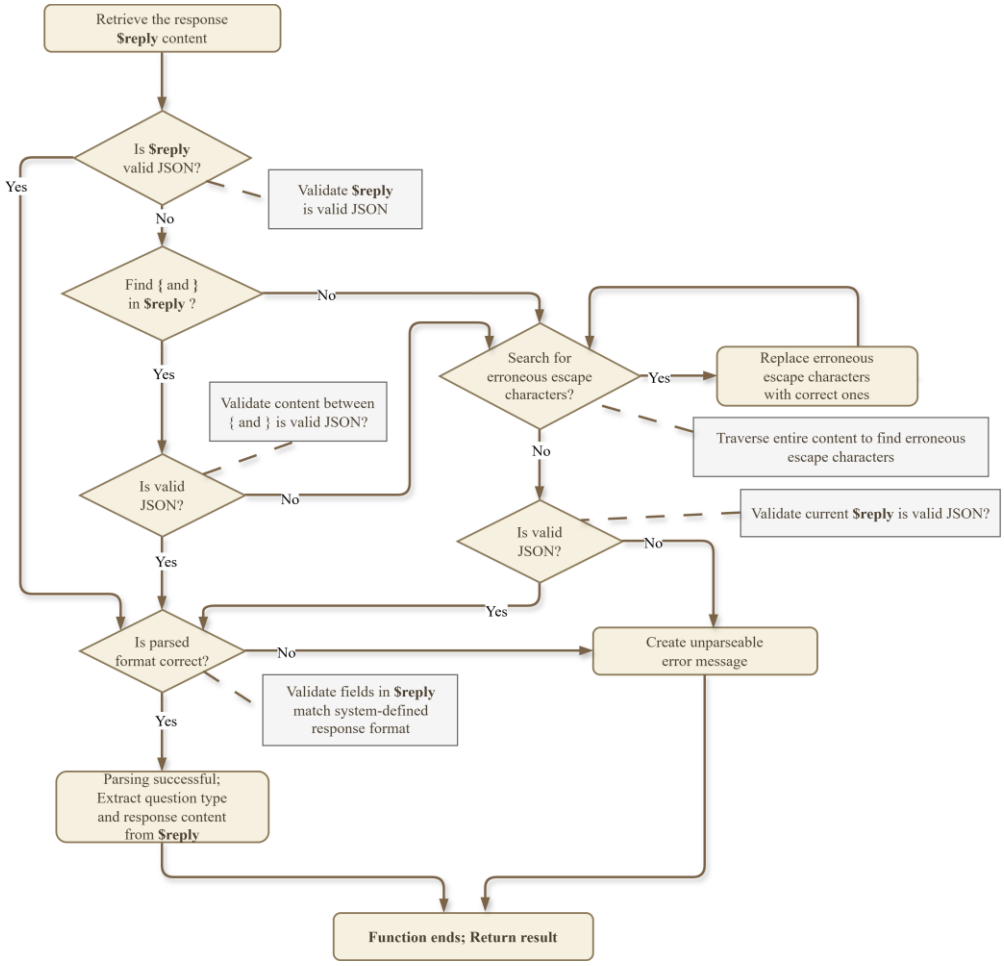


Figure 4: Post-processing Module

4.3 Threat Intelligence Synthesis

In this stage, the IoC Extraction Module identifies and analyzes user-provided threat intelligence, extracting critical indicators. The module offers multiple identification capabilities selectable by users, including CVE identification, attacker IP and domain name attribution, and APT attack lifecycle inference.

Users can directly import technical forums, articles, analytical reports, or dark web intelligence into the system without prior organization. The question recommendation module then lists appropriate query types for users to select, triggering LLM-based data extraction and analysis.

While modern LLMs possess broad general knowledge, research in Section 2.4 indicates that models may fail to correctly identify vulnerabilities not encountered during training. To ensure classification accuracy, the system leverages a knowledge base module to provide relevant domain knowledge. For example, since CWE contains over 900 software flaw categories, the system provides complete CWE documentation to the LLM during queries to prevent hallucinations.

Table 4 illustrates the prompt structure for CWE Identification as an example. The prompt begins with a concise context description, specifies the items to identify, provides a concrete response exemplar with placeholder data and format explanations, and concludes with an empty standard response template.

Table 4: IoC Extraction Module – CWE Identification

Item	Content
Type	Vulnerability Identification
Name	CWE Identification
Prompt Content	Based on the provided threat intelligence, which vulnerability in the system did the hacker exploit to attack the victim? Please list one CWE ID (if multiple exist, select the most significant one). Example: `{"CWE":{"item":"CWE-20: Improper Input Validation", "description":"The product does not validate or incorrectly validates input..."}}` Format: `{"CWE":{"item":"","description":""}}`
Return ID	CWE
Attachments	cwe_latest.pdf

4.4 Scenario Provisioning

4.4.1. Virtual Machine Provisioning

The Machine Deployment Module integrates VM configuration and virtual network orchestration, enabling users to complete attacker and victim infrastructure deployment with system assistance. The module adopts a pluggable design, allowing users to substitute underlying virtualization platforms. The default implementation communicates with OpenNebula via its XML-RPC API.

Following OpenNebula's operational logic, the system first identifies VM templates and then instantiates VMs from these templates. Several baseline templates are pre-configured (e.g., Windows Server 2022 Standard, Kali Linux 2024.2). The system supports multi-user access with OpenNebula's native user isolation, and integrates with LDAP for centralized permission management.

To accurately reflect conditions described in threat intelligence, the module allows per-machine network configuration, matching VM IP addresses to the Public IP or Private IP specified in intelligence reports. Virtual networks are managed by the OPNsense firewall, with DNS handled via Unbound for domain name-based communication. Users access VMs through the built-in WireGuard VPN service.

The primary functions for interfacing with the OpenNebula platform are:

- **`virtual_machine_template_list()`:** Retrieves available VM templates, filtering out redundant parameters and returning only the critical fields defined in Table 5 to avoid confusing the LLM.
- **`instantiate_virtual_machine()`:** Instantiates all VMs within a scenario simultaneously, taking vm_array and scenario_id as parameters. The system uses `parse_llm_vm_template()` to validate fields against Table 5 specifications. VMs are created with administrator privileges; once successfully deployed, permissions are modified to grant user ownership, preventing interference during the provisioning process.

Table 5: Return Parameters of the `virtual_machine_template_list()` Function

Field	Description
`TEMPLATE_ID`	The unique ID of the VM template in the system; required during VM creation.
`TEMPLATE_NAME`	The name of the VM template.
`NAME`	The name of the VM. Returned as empty, to be populated by the LLM.
`CPU`	Number of CPU cores for the VM.
`MEMORY`	Memory capacity of the VM.
`MEMORY_UNIT_COST`	Memory unit (e.g., MB, GB, TB).
`OS_ARCH`	Operating system architecture (e.g., x86_64, i686).
`DESCRIPTION`	The purpose of the VM. Returned as empty, to be populated by the LLM.
`ROLE`	The role of the VM. Returned as empty, to be populated by the LLM.

- **`change_virtual_machine_configuration()`:** Modifies VM configuration files via the OpenNebula Contextualization Add-on, pre-installed on all system VMs. This package enables direct hardware and software configuration through the platform by adding a virtual CD-ROM device containing a context.sh file with `KEY='VALUE'` pairs. Changes to the CONTEXT field trigger a virtual disk hot-plug event within 30–60 seconds, prompting the guest OS to apply updated settings.

Table 6 illustrates the prompt used for creating a basic attacker-victim machine setup, including situation description, VM template exemplars, and field specifications.

Table 6: Machine Deployment Module – Create Attacker and Victim Machines Only

Item	Content
Type	Environment Machine Provisioning
Name	Create Scenario Machines - Attacker and Victim Only
Prompt Content	To reproduce this attack, what kind of machine environment is needed to simulate the attacker and victim? Please refer to the following VM templates to select the appropriate machines. If the required resources exceed current limits, you may modify default values. In the NAME field, identify the machine as "attacker" or "victim" with a suffix (e.g., attacker-1). Provide additional details in the DESCRIPTION field. Please reply using the following JSON format; do not include other text or markdown syntax. VM Templates: `{{ vm_template_info }}`
Return ID	VMS
Attachments	None

4.4.2. Automated Software Installation

Once VM provisioning is complete, the Vulnerability Environment Installation Module assists with software deployment by importing structured data from the IoC Extraction Module and analyzing it via LLMs. The module features automated generation of installation scripts, enabling streamlined software setup that may require user feedback for error correction.

- **VM Software Installation:** This query provides the LLM with all machines in the scenario, victim machine lists, and response format exemplars. The LLM determines each machine's role from its description field and specifies software names, versions, and detailed installation steps. The prompt emphasizes that software must correspond directly to the threat intelligence to avoid installing unrelated components. The response follows a structured JSON format containing machine names mapped to arrays of software entries with name, version, installation method, and notes fields.
- **VM Software Installation Script:** Extends the previous query by generating executable scripts. The system detects the guest OS environment: PowerShell for Windows, Shell script for Linux. Based on OpenNebula's design, the VM context includes a start script field; when its content changes, a contextualization event triggers execution with root/administrator privileges.
- **VM Software Installation Script (with Error Messages):** A post-processing error-handling mechanism. When a previously generated script fails, the system submits the error message to the LLM for analysis and script revision. Users simply paste the error message displayed during the failed installation into the system's interactive response box.

4.5 LLM-Assisted Attack Script Generation

After completing environment provisioning, the Automated Attack Module integrates extracted IoC parameters to generate detailed attack methodologies. When the original intelligence lacks specific exploit details, the system may require users to supplement publicly available proof-of-concept code to produce a functional script. The module incorporates current machine configurations into the LLM's context, enabling it to draft attack scripts based on the real-time environment.

- **Attack Tactics:** Analyzes threat intelligence to infer potential attack vectors and provides users with a detailed, Markdown-formatted description of the attack workflow.
- **Automated Attack Script:** Generates Python attack scripts designed for autonomous execution, though the generation process itself may involve iterative refinement with user feedback. The prompt provides the LLM with detailed machine configurations including OS data and machine roles. As shown in Table 7, the script must be fully autonomous, requiring no manual intervention. The response format comprises three core fields: the script content, the designated attacker machine, and the target victim machine(s). Internal system information (such as IP addresses) is synchronized through the Post-processing Module, allowing users to execute scripts directly within the virtualized environment.

Table 7: Automated Attack Module – Python Attack Script Generation

Item	Content
Type	Automated Attack Script Generation
Name	Python Attack Script Generation

Prompt Content	Please initiate an attack against the victim's machine. Compose a fully automated Python attack script that can execute on the attacker's machine without requiring any manual parameters. Ensure that the IP information for both the attacker and the victim is correct.
	Python Version: 3.11
	Machine Configurations:
	`{{ vms_info }}`
	Please provide the response in JSON format.
	- attacker: Name of the attacker machine.
	- victim: Name of the victim machine.
	- script: The fully autonomous, executable script. Do not include commands that cannot be executed automatically.
	Example:
	`{"ATTACK_SCRIPT":{"attacker":"", "victim":"", "script":""}}`

Return ID	ATTACK_SCRIPT
Attachments	None

5 Experimental Results

This section describes the practical deployment of the proposed system through two attack scenarios to evaluate its effectiveness.

5.1 CVE-2024-4577 Simulation

CVE-2024-4577, disclosed by DEVCORE's Orange Tsai in June 2024 [17], allows attackers to execute arbitrary PHP code on vulnerable servers. The vulnerability exploits the Windows Best-Fit character encoding conversion behavior: when a Unicode character cannot be represented in the current code page, the system converts it to the closest matching character. PHP's design overlooked this behavior, enabling attackers to bypass security restrictions using specific character sequences. The vulnerability affects all PHP versions released before June 2024 when configured in CGI mode.

5.1.1. Attack Reproduction

The user imported the vulnerability description from the DEVCORE blog [17] as a text file into the system, creating a new scenario for CVE-2024-4577.

Stage 1: Threat Intelligence Analysis. The IoC Extraction Module analyzed the imported data and produced the results summarized in Table 8. The system identified the vulnerability as OWASP Top 10 A03:2021 Injection, classified it as CWE-94 (Improper Control of Generation of Code), and enumerated affected PHP versions: PHP 8.3 < 8.3.8, PHP 8.2 < 8.2.20, PHP 8.1 < 8.1.29, and all XAMPP for Windows versions.

Table 8: System-Inferred Cause and Affected Software of CVE-2024-4577

Query	System Response
Vulnerability Description	The root cause of this vulnerability is that PHP overlooked the Windows Best-Fit character encoding conversion behavior during its design, allowing unauthenticated attackers to bypass CVE-2012-1823 protection through specific character sequences and perform parameter injection to achieve remote code execution. This vulnerability affects all PHP versions installed on Windows, particularly under CGI execution mode or when the PHP executable is exposed in CGI directories.
OWASP Top 10	A03:2021 – Injection. This attack exploits the parameter handling vulnerability in PHP CGI mode to inject malicious commands. This method falls under injection attacks (Injection), including SQL, NoSQL, command, LDAP, and other injection types. Attackers craft special input to trick applications into executing unauthorized commands or queries, thereby obtaining sensitive data, tampering with data, or damaging the system.
CWE	CWE-94: Improper Control of Generation of Code ('Code Injection'). The CGI mode in PHP fails to properly control character encoding conversion, allowing unauthenticated attackers to execute arbitrary code on remote servers through parameter injection.

Stage 2: Scenario Construction. The system configured two VMs as shown in Table 9: an attacker host (`attacker-1`) running Kali Linux and a victim host (`victim-1`) running Windows Server 2022. The Vulnerability Environment Installation Module then identified that `victim-1` requires PHP 8.2.19 and Apache HTTP Server 2.4.54.

For PHP installation, the system generated a PowerShell script for automated PHP installation on `victim-1`, covering download, extraction, environment variable configuration, and verification. The initial script contained an incorrect download URL, which the user reported through the system's error feedback mechanism. The system then regenerated a corrected script with the proper URL (windows.php.net), and PHP 8.2.19 was successfully installed.

For Apache installation, the system generated a comprehensive PowerShell script that downloads the Apache 2.4.54 package, extracts and reorganizes the directory structure, registers the Apache service, configures CGI mode integration with PHP including proper file handler settings and directory permissions, and restarts the service.

Table 9: Attacker and Victim Machines Automatically Created by the Machine Deployment Module

Machine	Template_ID/ Template_NAME	CPU	Memory	Role	Description
`attacker-1`	3/Kali Linux 2024.2	2	4096 MB	Attacker	Penetration testing environment
`victim-1`	1/Windows Server 2022 Standard	2	4096 MB	Victim	Vulnerable PHP web server

Stage 3: Attack Script Generation. The system's Automated Attack Module produced an attack chain analysis, as shown in Table 10, detailing the exploitation of php-cgi.exe through parameter injection using allow_url_include and auto_prepend_file.

When provided only with the DEVCORE blog content, the LLM generated an attack script that directly used the "-" character as a CGI parameter. This approach would only trigger the older CVE-2012-1823 vulnerability, which has already been patched in PHP 8.2.19, rendering the attack unsuccessful against the target environment.

After the user supplemented publicly available exploit scripts [18] [19], the system generated a revised Python script incorporating the following key elements: (1) HTTP headers declaring form-format content for proper PHP interpretation; (2) PHP CGI parameters including `cgi.force_redirect=0`, `disable_functions=""`, `allow_url_include=1`, and `auto_prepend_file=php://input`; (3) critically, the "-" character is replaced with the encoded string "`%AD`" to trigger the Best-Fit behavior—on a Traditional Chinese Windows system (Big5 encoding, code page 950), Unicode U+00AD (Soft Hyphen) is converted to U+002D (Hyphen-minus), thereby triggering CVE-2024-4577; (4) a PHP payload executing the system command. The script successfully attacked the victim host, and replacing the payload with "`whoami`" confirmed the attacker obtained nt authority\system privileges.

Table 10: System-Inferred Attack Chain for the CVE-2024-4577 Scenario

Step	Description
1. Verify PHP CGI Configuration	Confirm PHP runs in CGI mode on the victim server with proper handler and action directives in httpd.conf.
2. Construct Malicious Request	Craft a URL targeting `php-cgi.exe` with parameter injection to enable `url_include` and `auto_prepend_file`.
3. Send Malicious POST Request	Send a POST request containing PHP code (e.g., `system("ipconfig")`) to execute arbitrary commands on the victim server.
4. Post-exploitation	Modify the payload to execute additional commands, download malware, or establish persistent control.

5.1.2. Experimental Summary

The system successfully assisted users in reproducing the CVE-2024-4577 attack scenario from a single threat intelligence source. In the final attack script generation stage, users can supplement publicly available exploit code to enable the system to generate customized attack scripts tailored to the target environment.

5.2 APT TAG-70 Simulation

TAG-70 (also known as Winter Vivern, TA473, UAC-0114) is a pro-Russian APT organization active since December 2020, primarily targeting government agencies in Eastern Europe. In 2023, the group exploited CVE-2023-5631 in Roundcube Webmail to compromise systems of the Ukrainian Armed Forces and the Georgian Ministry of Defence. This study uses the threat intelligence report published by Recorded Future [20] to construct a simulated enterprise network environment.

5.2.1. Attack Reproduction

We simulated the workflow of enterprise security analysts by importing the Recorded Future PDF report directly into the system.

Stage 1: Threat Intelligence Analysis. The IoC Extraction Module identified CVE-2023-5631, a cross-site scripting (XSS) vulnerability in Roundcube Webmail, classifying it as OWASP Top 10 A03 Injection. The system also generated the APT attack lifecycle, detailing each phase from reconnaissance through data exfiltration. In the reconnaissance phase, TAG-70 conducted phishing campaigns from IP address 38.180.76.31, sending emails with malicious links or attachments to collect target credentials. For initial access, the group exploited CVE-2023-5631 in the Roundcube Webmail server from 176.97.76.129, gaining access when the victim merely opened the infected email. During the execution phase, TAG-70 deployed JavaScript-based malware from 38.180.76.31 to capture email credentials and transmit them to the C2 server at 198.50.170.72. For command and control, TAG-70 used Tor to conceal its location, communicating through non-standard ports such as TCP 443. In the data exfiltration phase, the group collected and exfiltrated email content and credentials from the compromised Roundcube server, primarily targeting government and military accounts. The ultimate impact involved the attacker logging the victim out of Roundcube and displaying a spoofed login page from 176.97.66.57 targeting mail.mod.gov.ge; when the victim entered credentials, they were transmitted to the C2 server, enabling access to sensitive military and government emails.

The analysis revealed that the attacker initially used IP address 38.180.76.31 for reconnaissance with the malicious domain recsecas.com, which resolved to 38.180.76.31 before September 2023 and changed to 176.97.76.129 afterward, indicating deliberate IP switching to evade tracking. The attacker also employed Tor network nodes to conceal C2 communications.

When analyzing APT attack lifecycles, the system can describe each phase based on threat intelligence fields. However, since reports often contain multiple attack events targeting different countries with different C2 IP addresses, the system may have difficulty precisely distinguishing relationships between these IPs without explicit specification of a particular attack event. This requires multiple iterative queries to obtain clear responses.

Stage 2: Environment Construction. To provide a realistic enterprise environment, we used the system's enterprise environment deployment feature to automatically generate a complete network topology, including organizational units (HR, finance, marketing), internal services (web servers, DNS servers), and attacker infrastructure (workstations, C2 servers). The generated topology comprised 20 VMs spanning multiple network segments. The system configured VM network IP addresses according to the threat intelligence and utilized the DNS server to resolve domain names to designated IP addresses, avoiding resolution failures caused by attackers removing DNS records in the real internet environment.

The system installed Roundcube Webmail version 1.5.2 containing the CVE-2023-5631 vulnerability on the HR-EmailServer host, accessible at https://mail.mod.gov.ge.

Stage 3: Attack Reproduction. For authenticity, we placed actual malicious scripts used by the attackers into the scenario. As shown in Figure 5, the attack flow proceeds as follows: the attacker sends a phishing email to the victim; upon opening the email, malicious code connects to the `c2-1` machine to retrieve the `checkupdate.js` script, which executes in the victim's browser and awaits further C2 commands.

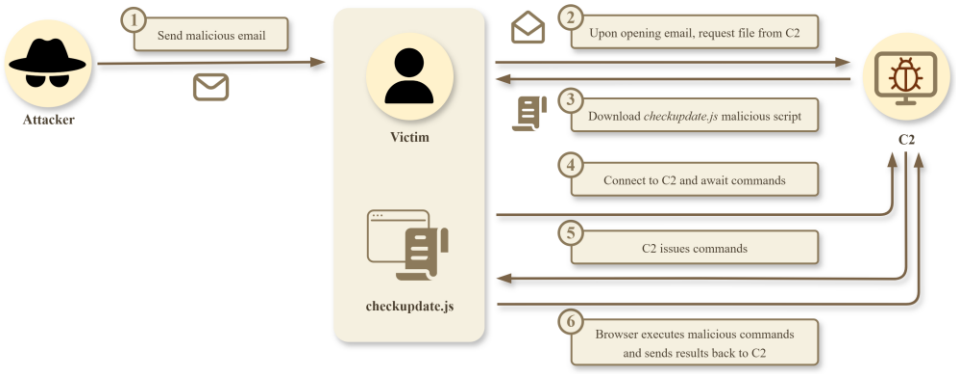


Figure 5: Attack Flow Against the Roundcube Webmail System

Using SHA-256 hashes from the report's IoC appendix, we located malicious script samples from online malware databases. The sample with SHA-256 hash beginning with `ea22b3e9` transmits the victim's Roundcube username and password to the C2 domain hitsbitsx.com. When simulating the victim browsing email on the `hr-1` VM, the malicious code logged the user out of Roundcube and displayed a spoofed login page. Upon entering credentials, the script's onClickSendCredentials function transmits them to the `c2-1` machine.

We also obtained a variant checkupdate.js script (SHA-1 beginning with `97ed59`) identified in a separate ESET threat intelligence report. This script silently exfiltrates all inbox emails to the `c2-1` machine (`domain: recsecas.com`) without any visible indication to the victim.

5.2.2. Experimental Summary

The system effectively assisted in reproducing the APT TAG-70 attack scenario by rapidly parsing the APT attack lifecycle, systematically enumerating attacker behaviors at each phase, and automatically constructing the required virtual environment including domain name and IP address configurations that closely match the actual attack scenario.

5.3 Discussion and Comparison

In the preceding sections, we demonstrated how the system assists security analysts in step-by-step attack reproduction through modular design—from key information extraction, to realistic environment construction, to attack script validation and execution. The system is applicable to both common CVE vulnerabilities and large-scale APT campaigns.

Regarding IoC extraction, Lee et al. [21] proposed the PrioriC framework, which uses ChatGPT to analyze 116 threat intelligence reports, extracting critical information and classifying attack techniques and targeted assets. Liu et al. [22] utilize ChatGPT to construct STIX topologies from threat intelligence, with post-processing scripts using regular expressions to capture indicators not initially identified by the model.

As shown in Table 11, while all approaches can extract IoCs from threat intelligence, our system leverages LLMs to comprehensively evaluate a broader range of indicators, providing additional contextual information such as APT attack lifecycles and attacker activity timelines for more comprehensive analysis.

Table 11: Comparison of IoC Extraction Capabilities Between the Proposed System and Other Frameworks

Feature	Proposed System	PrioriC Framework	Liu et al.
CVE ID	✓	✓	✓
CWE ID	✓	-	-
OWASP Top 10 Classification	✓	-	-
IP Address	✓	-	✓
Domain Name	✓	-	✓
Victim Industry Category	✓	✓	-
Victim Role	✓	-	✓
Victim Operating System	✓	✓	-
Victim Software	✓	-	✓
Attacker Techniques Used	✓	✓	✓
Attacker Network Protocols Used	✓	✓	-
Attacker-Exfiltrated Victim Files	✓	-	✓
Attack Activity Timeline	✓	-	-
Attack Lifecycle	✓	-	-

Regarding attack script generation, as shown in Table 12, both PentestGPT and our system support diverse attack types. However, PentestGPT operates as a conversational chatbot, requiring users to manually provide necessary information and attack methods. HackingBuddyGPT can automatically determine attack methods based on machine information but is currently limited to specific scenarios such as Linux privilege escalation and web penetration testing. A key advantage of our system is its ability to automatically collect attacker and victim environment information and provide it to the LLM, eliminating manual input and ensuring data accuracy.

Furthermore, the proposed system provides an integrated environment where users can construct attack scenarios and automatically generate attack scripts based on the current environment state. Traditional manual attack reproduction often requires weeks; with our system's guidance, multiple attack scenarios can be reproduced within a single day.

Table 12: Comparison of Attack Script Generation Capabilities

Feature	Proposed System	PentestGPT	HackingBuddyGPT
Diverse Attack Types	✓	✓	-
Automated Prompt Generation	✓	-	✓
Automated Attack Method Selection	✓	-	✓

Limitations. It is important to note that the primary contribution of this work is the proposal of an end-to-end framework that integrates LLM-driven threat intelligence extraction, virtual environment provisioning, and attack script generation into a unified platform. The current evaluation is based on representative case studies to demonstrate the feasibility and practical applicability of this framework.

One key challenge lies in the inherent non-determinism of LLM outputs. Given identical prompts, the model may produce structurally or semantically different responses across runs, making consistent

quantitative measurement difficult without first establishing a standardized evaluation methodology. To date, no widely accepted benchmark exists for evaluating end-to-end attack reconstruction systems. Existing LLM-assisted security research, such as HackingBuddyGPT [9] and PentestGPT [10], and primarily relies on Capture-the-Flag (CTF) platforms [11] [12] where success is measured by binary flag retrieval, a metric that does not directly transfer to the multi-stage reconstruction workflow addressed in this work.

Additionally, as observed in Section 5.1, the initial attack script generated by the system failed to trigger the target vulnerability and required user-supplied exploit references for correction. This reflects a deliberate design choice in the current framework: the system is intended to operate as an LLM-assisted, human-in-the-loop tool rather than a fully autonomous agent. The error feedback mechanism described in Section 4.4 provides a structured pathway for iterative refinement, and characterizing the frequency and nature of such interventions across a broader set of scenarios remains an important direction for future investigation.

6 Conclusions and Future Works

This research proposes a novel system architecture that integrates LLM technologies to assist security analysts in performing threat intelligence analysis and vulnerability reproduction. Users can input various forms of attack intelligence, such as threat intelligence reports from international security organizations or proof-of-concept scripts shared by security communities, and the system assists and guides them through the entire process from threat analysis, attack environment construction, to vulnerability reproduction and verification, with human oversight at key decision points.

Current research on applying LLMs in cybersecurity mainly focuses on threat detection, with relatively little work exploring their use in threat intelligence analysis. The proposed system leverages the powerful analytical capabilities of LLMs to help analysts more efficiently understand attack events, extract key attack indicators from reports with high efficiency, and construct simulated attack environments for deeper analysis of the attack process.

The system features a robust virtualization architecture that allows users to deploy virtual environments directly within the system and conduct experiments in network-isolated environments. Through the automated attack script generation functionality, the system produces scripts tailored to specific scenarios, enabling users to gain a detailed understanding of the entire attack lifecycle. The modular design architecture provides strong scalability and flexibility for future development. As AI technologies advance, the modular LLM interface can flexibly integrate emerging models, eliminating single-model dependency and enabling more accurate threat analysis.

Acknowledgments

This work was supported in part by the National Science and Technology Council of Taiwan under Grant NSTC 114-2634-F-110-001-MBK, in part by the Information Security Research Center at National Sun Yat-sen University, and in part by the Intelligent Electronic Commerce Research Center from the Featured Areas Research Center Program through the Framework of the Higher Education Sprout Project by the Ministry of Education in Taiwan.

References

[1] Canadian Centre for Cyber Security. (2024). National cyber threat assessment 2025–2026. https://www.cyber.gc.ca/en/guidance/national-cyber-threat-assessment-2025-2026

[2] IBM. (2025). Cost of a data breach report 2025. https://www.ibm.com/reports/data-breach

[3] Parmar, M., & Domingo, A. (2019). On the use of cyber threat intelligence (CTI) in support of developing the commander's understanding of the adversary. In MILCOM 2019 – IEEE Military Communications Conference. IEEE.

[4] MITRE Corporation. (n.d.). ATT&CK. Retrieved April 8, 2026, from https://attack.mitre.org/

[5] Mundt, M., & Baier, H. (2023). Threat-based simulation of data exfiltration toward mitigating multiple ransomware extortions. Digital Threats: Research and Practice, 4(4).

[6] Karagiannis, S., Tokatlis, A., Pelekis, S., Kontoulis, M., Doukas, G., Ntanios, C., & Makos, E. (2022). ATT&CK documentation, emulation and mitigation operations. In Proceedings of the 26th Pan-Hellenic Conference on Informatics (PCI 2022). ACM.

[7] Iturbe, E., Arcas, J., Rios, E., & Toledo, N. (2024). A multi-layer approach through threat modelling and attack simulation for enhanced cyber security assessment. In Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES 2024). ACM.

[8] MITRE. (n.d.). Common vulnerabilities and exposures (CVE). Retrieved April 8, 2026, from https://cve.mitre.org/

[9] Happe, A., & Cito, J. (2023). Getting pwn'd by AI: Penetration testing with large language models. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023) (pp. 2082–2086). ACM.

[10] Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., & Ras, S. (2024). PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In Proceedings of the 33rd USENIX Security Symposium. USENIX Association.

[11] Hack The Box. (n.d.). Hack The Box. Retrieved April 8, 2026, from https://www.hackthebox.com/

[12] VulnHub. (n.d.). VulnHub. Retrieved April 8, 2026, from https://www.vulnhub.com/

[13] Lin, J., & Mohaisen, D. (2025). From large to mammoth: A comparative evaluation of large language models in vulnerability detection. In Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS 2025). Internet Society.

[14] Lekssays, A., Mouhcine, H., Tran, K., Yu, T., & Khalil, I. (2025). LLMxCPG: Context-aware vulnerability detection through code property graph-guided large language models. In Proceedings of the 34th USENIX Security Symposium. USENIX Association.

[15] Sheng, Z., Chen, Z., Gu, S., Huang, H., Gu, G., & Huang, J. (2025). LLMs in software security: A survey of vulnerability detection techniques and insights (arXiv:2502.07049). arXiv. https://arxiv.org/abs/2502.07049

[16] Happe, A., Kaplan, A., & Cito, J. (2024). LLMs as hackers: Autonomous Linux privilege escalation attacks (arXiv:2310.11409). arXiv. https://arxiv.org/abs/2310.11409

[17] DEVCORE. (2024, June 6). Security alert: CVE-2024-4577 PHP CGI argument injection vulnerability. DEVCORE Blog. Retrieved July 30, 2024, from https://devco.re/blog/2024/06/06/security-alert-cve-2024-4577-php-cgi-argument-injection-vulnerability

[18] K3ysTr0K3R. (n.d.). CVE-2024-4577 PHP CGI argument injection remote code execution [Computer software]. GitHub. Retrieved April 9, 2026, from https://github.com/K3ysTr0k3R/CVE-2024-4577-EXPLOIT

[19] Chocapikk. (n.d.). CVE-2024-4577 PHP CGI argument injection [Computer software]. GitHub. Retrieved April 9, 2026, from https://github.com/Chocapikk/CVE-2024-4577

[20] Recorded Future. (2024, February 28). Russia-aligned TAG-70 targets European government and military mail servers in new espionage campaign. Insikt Group. Retrieved April 8, 2026, from https://www.recordedfuture.com/research/russia-aligned-tag-70-targets-european-government-and-military-mail

[21] Lee, S., Shieh, S. W., & Tsai, M. (2024). Enterprise-centric intelligence: A prioritization scheme for cyber threat intelligence and common vulnerabilities and exposures. Computer, 57(7), 141–152.

[22] Liu, J., & Zhan, J. (2023). Constructing knowledge graph from cyber threat intelligence using large language model. In Proceedings of the 2023 IEEE International Conference on Big Data (BigData 2023). IEEE.